

Traivo One — Analys: Presentation vs. Faktisk Implementation

Analyserad: 2026-05-13

Presentation: Utgångspunkt Produktion och Team Kinab.pptx (6 slides)

Repository: github.com/Patron1965/traivo-one (<https://github.com/Patron1965/traivo-one>) (main branch)

Kodbas: ~221 000 rader TypeScript/TSX, 148 databastabeller, 108 sidor/vyer

Executive Summary

Presentationen (Kinab) beskriver en **operativ planeringsfilosofi** för fältserviceverksamhet — daglig och veckovis planering av kärltvättar med fokus på lönsamhet, balans och uppföljning. Den är inte en teknisk systemspecifikation utan en **verksamhetsvision** för hur arbetet ska organiseras.

Traivo One-kodbasen överstiger i nästan alla avseenden det som presentationen antyder.

Systemet är en fullständig enterprise-fälttjänstplattform med 108 sidor, OR-Tools/ALNS-ruttoptimering, AI-integration (OpenAI), kundportal, Fortnox-integration, IoT-stöd och mycket mer. Presentationens koncept — daglig produktionsmåluppföljning, veckoplanering, resursbalans och enhetsansvar — finns implementerade som kärnfunktioner, men systemet sträcker sig långt bortom det som presenteras.

Sammanfattning i siffror

Dimension	Presentation	Implementation
Daglig planering	✓ Konceptuellt beskrivet	✓ DayTimelineView, DailyProgressCard, ruttop-timering
Veckoplanering	✓ 40h-modell beskriven	✓ WeekPlanner med drag-and-drop, What-If, constraint layers
Ruttop-timering	✗ Inte nämnt explicit	✓ OR-Tools CVRPTW + ALNS metaheuristik, Geoapify VRP
Produktionsmål & KPI	✓ Kärldag-mål, plan vs utfall	✓ KPI-dashboards, trendanalys, AI-insights
Enhetsansvarig-roll	✓ Detaljerat beskrivet	⚠ Rollsystem finns (admin/planner/user) men inte specifik "enhetsansvarig"-roll
Avvikelsehantering	✓ 4-stepsprocess beskriven	✓ AnomalyAlerts, avvikelse-detektering, disruption service
Veckomöte & uppföljning	✓ Agenda beskriven	✓ WeeklyReport-scheduler, rapporterings-dashboards
AI/Automation	✗ Inte nämnt	✓ OpenAI-integration, AI-planner, prediktiv analys
Multi-tenant (SaaS)	✗ Inte nämnt	✓ Fullständig tenant-isolering med middleware
Kundportal	✗ Inte nämnt	✓ 12+ portal-sidor med bokningar, fakturor, kontrakt
Fakturering/Fortnox	✗ Inte nämnt	✓ Kompletta Fortnox-integration
Mobilapp (fält)	✗ Inte nämnt	✓ Offline-first mobilapp med fokusläge, signatur, QR

1. Presentation Overview — Vad som utlovas

Presentationen består av 6 slides och är varumärkt **Kinab — Återvinningsmiljöexperten**. Den beskriver en operativ modell, inte ett IT-system. Huvudbudskapen:

Slide 1: Introduktion

- Kinab är rikstäckande inom tvätt och drifthållning av avfallssystem.

Slide 2: “Nå vår plan — för lönsamhet och planering”

- **Produktionsmål:** 100 kärl/dag
- **Lönsamhetsgräns:** ~10% marginal, vinst skapas först efter ~90 kärl
- **Break-even-scenario:** 85 kärl = förlust, 90 kärl = nollresultat, 100+ kärl = vinst
- **Carry-over-effekt:** 10 missade kärl dag 1 → 110 kärl dag 2 → stress, risk
- **Budskap:** “När vi når vår plan varje dag — skapar vi lönsamhet, stabilitet och ett hållbart företag.”

Slide 3: “Så planerar vi en arbetsdag — Produktion och resor”

- **5 planeringspusseldelar:**
 1. Jobbet byggs av uppgifter (kärl × tid + rum-tvätt = total produktionstid)
 2. Flera jobb under dagen (A-D på olika adresser)
 3. Restid mellan jobb (15–25 min)
 4. Tidsfönster hos kund (t.ex. “ej efter 15:00”)
 5. Pusslet läggs — optimering baserat på tid, rutter, arbetsbelastning
- **Kvalitetsmål:** Rätt tid per uppgift, korta rutter, leverans i tid, nå dagsmålet
- **Resultat:** Mer produktionstid, mindre restid, lönsam verksamhet

Slide 4: “Veckovy — Så bygger vi en hållbar arbetstid”

- **40-timmarsmodell:**
- 28h planerad produktion & restid
- 12h reserverad tid (rast 45 min, tankning 30 min, buffert 30 min, service 15 min, admin 15 min)
- **Övertidshantering:** Max 44h, med varning — “Återkommande överskridande är inte hållbart”
- **Motto:** “Rätt tid på rätt plats i rätt tempo — varje vecka”

Slide 5: “Enhetsansvarig Produktionsenhet”

- **Tre ansvarsområden:** Team, Fordon, Uppdrag
- **Uppföljning (plan vs utfall):** Tabell med antal kärl, stopp, km, avvikelser, produktions-/restid
- **Avvikelseprocess:** Identifiera → Analysera → Prioritera → Åtgärda
- **Veckoplaneringsmöte:** Resultat, trender, planering, kvalitetsmål, beslut
- **Kvalitetsmål:** Kundnöjdhet, säkerhet, miljö, effektivitet, kontinuerlig förbättring

Slide 6: Avslutning

- “Samarbete med helhetsperspektiv” — kontaktuppgifter (kinab.se)
-

2. Implementation Status — Vad som finns i koden

2.1 Teknisk stack

Lager	Teknologi
Frontend	React 18, TypeScript, Vite, Tailwind CSS, shadcn/ui, Recharts, Leaflet
Backend	Express.js, Node.js, Zod-validering
Databas	PostgreSQL, Drizzle ORM (148 tabeller, 79 migrationer)
AI	OpenAI (gpt-4o-mini), AI-persona-system
Optimering	Python FastAPI-mikrotjänst med OR-Tools CVRPTW + ALNS
Geokodning	Geoapify, Google Geocoding, OpenStreetMap Nominatim
Routing	Geoapify Routing API, OSRM
Realtid	Socket.IO (WebSocket)
Fakturering	Fortnox API-integration
SMS/Notiser	Twilio, Resend (e-post)

2.2 Kärnfunktioner relaterade till presentationen

✓ Daglig planering & produktionsmål

- **DailyProgressCard.tsx** — Visuell progress mot dagsmål (kärl tvättade/planerade) med celebration-animation vid 100%
- **DayTimelineView** (i weekplanner/) — Tidslinjevy för en dag med jobb placerade i tid
- **DayReport.tsx** — Daglig rapport med sammanställning
- **TodayOverview.tsx** — Dashboard-komponent som visar dagens status
- **AutoDistributeToday.tsx** — Auto-fördelning av dagens arbeten

✓ Veckoplanering

- **WeekPlanner.tsx** (+ ~15 underkomponenter i weekplanner/) — Avancerad drag-and-drop veckoplanerare med:
 - WeekGridView, DayTimelineView, MonthView, RouteMapView
 - UnscheduledSidebar (icke-schemalagda ordrar)
 - WhatIfPreview (konsekvensanalys)
 - DisruptionPanel (störningshantering)
 - PlannerToolbar med filter, zoom, densitet
 - Constraint-overlays (synliga begränsningar)

- Conflict detection (restidsmedveten)
- Bulk-schemaläggning och tilldelning

✓ Ruttoptimering

- `optimization-service/main.py` — Python FastAPI-tjänst med OR-Tools CVRPTW
- `optimization-service/alns.py` — ALNS (Adaptive Large Neighborhood Search) + Local Search (2-opt, or-opt) postoptimering
- `server/route-optimizer.ts` — TypeScript-orchestrering av optimering
- `server/vrp-constraints.ts` (673 rader) — VRP-begränsningsmodellering (tidsfönster, kompetens, kapacitet, beroenden)
- `server/optimization-job-runner.ts` — Asynkron jobbkö för stora optimeringar (>30 stopp)
- `server/distance-matrix-service.ts` — Distansmatriser med caching
- Stöd för: tidsfönster, fordonkapacitet, kompetenskrav, orderberoenden, teambaserad optimering

✓ KPI:er & Uppföljning

- `server/routes/kpiRoutes.ts` (2429 rader) — Omfattande KPI-endpoints
- `Dashboard.tsx` — Interaktivt dashboard med Recharts-diagram (LineChart, BarChart, PieChart, ComposedChart)
- `DataQualityDashboard.tsx` — Datakvalitets-dashboard
- `CapacityForecastTab.tsx` / `CapacityOverview.tsx` — Kapacitetsprognos
- `ReportingDashboardPage.tsx` — Rapporteringsvy
- `server/weekly-report.ts` — Automatisk veckorapportgenerering (schemalagd)
- **KPI-beräkning i AI-planner:** `calculatePlanningKPIs()` inkluderar ställtider, resurseffektivitet, förseningar, kostnadsanomalier

✓ Avvikelsehantering

- `server/anomaly-monitor.ts` — Automatisk anomalidetektering
- `AnomalyAlerts.tsx` — UI för avvikelseaviseringar
- `server/disruption-service.ts` (538 rader) — Störningshantering
- `DeviationReports` (schema) — Dedikerad tabell för avvikelserapporter
- `server/services/sla-risk-engine.ts` — SLA-riskberäkningsmotor
- `SLARiskPanel.tsx` — UI-panel för SLA-risker

✓ Tidsfönster och kundpreferenser

- `task_desired_timewindows` — Tabell för önskade tidsfönster per uppgift
- `object_time_restrictions` — Tidsbegränsningar per objekt
- `deliveryPreferences` — JSON-fält på kund- och objektnivå med slottider och blockerade tider
- `TaskTimewindowsEditor.tsx` / `DeliveryPreferencesEditor.tsx` — UI-editorer
- VRP-constraints respekterar tidsfönster vid optimering

✓ Resurshantering & Team

- `resources` tabell med: typ (person/vehicle), kompetens, hemposition, GPS, PIN för mobilapp
- `teams` + `team_members` — Team-organisering
- `resource_availability` — Tillgänglighetsplanering
- `resource_vehicles` + `resource_equipment` — Koppling resurs → fordon/utrustning
- `work_sessions` + `work_entries` — Arbetstidsregistrering
- `ResourceList.tsx` / `ResourcesPage.tsx` — Resurshantering i UI

2.3 Funktioner BORTOM presentationen

Dessa finns i koden men nämns inte alls i presentationen:

Funktionsområde	Implementation
AI-driven planering	OpenAI-integration med persona-system, AI-suggestions, AI Field Assistant, AI Command Center
Multi-tenant SaaS	Fullständig tenant-isolering med middleware, rollbaserat åtkomstkontroll (platform-owner/admin/planner/user)
Kundportal	12+ sidor: dashboard, fakturor, kontrakt, kartor, ärendehantering, self-booking, verifiering
Fortnox-integration	Komplett faktureringsflöde med frysning av priser, export, kontraktmatchning
Mobilapp (Traivo Go)	Offline-first med fokusläge, signatur, materiallogg, QR-kod, GPS-spårning
Orderkoncept-wizard	9-stegswizard för att skapa ordermallar (objekt → artiklar → fakturering → leveransmodell)
Import/Datakvalitet	Avancerad importmodul med kolumnmapping, batch-watchdog, datakvalitets-dashboard
Auto-clustering (DBSCAN)	Geografisk klustring av objekt med parameteroptimering
Prediktiv analys	ML-fas 0/1 med LightGBM-kvantilprediktion, shadow/canary/active-modellcykel
IoT-integration	Signalmottagning, API-nycklar per enhet, dashboard
Telefoni/SMS	Twilio-integration, ETA-notiser, kundkommunikation
Inspektioner & Checklistor	Mallbaserade checklistor, inspektionsmetadata
Fleet Management	Fordonshantering, bränslelog, underhållslogg
Miljöcertifikat	Miljödata-rapportering
Proaktiv försäljning	ProactiveSalesPage
MCP-integration	Model Context Protocol-server för AI-agentanvändning



3. Gap Analysis — Diskrepanser

3.1 Presentationens koncept → Implementationsstatus

Presentationskoncept	Status	Detaljer
100 kärldag-mål	✓ Implementerat	DailyProgressCard visar completed/total med progress bar. Dock generisk (inte specifikt "kärldag"-fokuserad utan arbetsorder-baserad).
Break-even-visualisering (85/90/100 kärldag)	⚠ Delvis	Dashboarden visar KPI:er och trender, men den specifika break-even-visualiseringen (lönsamhetskurvan) som presentationen visar finns inte som en dedikerad vy. ROI-rapporter (ROIReportPage , LundstamsROIPage) finns men är mer generella.
Carry-over/kompensationseffekt	⚠ Indirekt	WeekPlanner hanterar icke-schemalagda ordrar via UnscheduledSidebar . Automatic carryover av missade jobb finns som koncept men inte som explicit "carry-over-varning" som presentationen beskriver.
5-pusselmodellen (jobb, tid, resa, tidsfönster, optimering)	✓ Implementerat	Alla 5 delar finns: jobb med uppgifter (work_orders + work_order_lines), multipla adresser, restidsberäkning (distance-matrix), tidsfönster (time_restrictions, delivery_preferences), VRP-optimering.
Produktionstid vs restiduppdelning	✓ Implementerat	Ruttoptimeraren rapporterar explicit totalDriveTime , totalWorkTime , timeSaved , distanceSaved , estimatedFuelSaved , estimatedCostSaved .
40-timmars veckomodell (28h + 12h)	⚠ Konfigurerbart	planning_parameters och resurstillgänglighet finns. Dock är den exakta 28h/12h-uppdelningen inte hårdkodad

Presentationskoncept	Status	Detaljer
		som en explicit modell utan hanteras via kapacitets- och tillgänglighetsplanering.
Enhetsansvarig-rollen	 Delvis	Rollsystemet har <code>platform-owner</code> , <code>admin</code> , <code>planner</code> , <code>user</code> . Det finns ingen specifik "enhetsansvarig"-roll . Den närmaste motsvarigheten är <code>planner</code> som kan se dashboards och KPI:er.
Plan vs utfall-tabell	 Implementerat	KPI-endpoints (<code>kpiRoutes.ts</code>) levererar data för kärl, stopp, km, avvikelser, tid. Dashboard visar dessa med trender. Veckorapporter genereras automatiskt.
Veckomöte-agenda	 Indirekt	Ingen explicit "veckomötesvy" eller agenda-generator, men all data som behövs för mötet (KPI:er, trender, avvikelser, planering) finns tillgänglig i dashboard och rapporter.
Kvalitetsmål (kundnöjdhet, säkerhet, miljö, effektivitet)	 Delvis	<code>annual_goals</code> -system, kundfeedback (<code>technical_ratings</code>), miljödata (<code>environmental_data</code>), men inte som en samlad "kvalitetsmål"-vy matchande presentationens modell.

3.2 Vad som finns påbörjat men inte färdigt

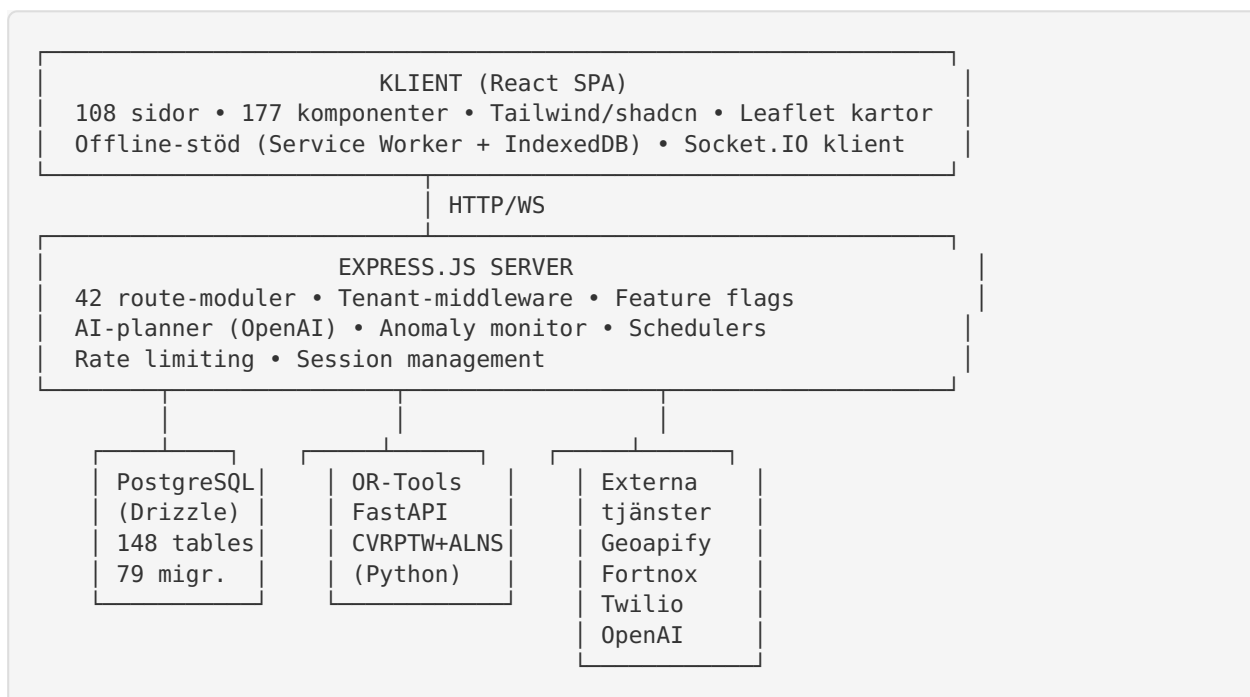
Område	Status	Bevis i koden
ML-prediktion	Fas 0-1, scaffolding	<code>mlPredictionClient.predictDurations()</code> returnerar alltid <code>null</code> tills modell finns. Feature snapshots skrivs. Träningsskript (<code>train_duration_model.py</code>) finns men kräver data.
OR-Tools optimization service	Implementerad men extern	<code>optimization-service/</code> med Dockerfile, men kräver separat deploy (Python FastAPI). Fallback till Geoapify VRP om tjänsten inte körs.
OSRM-integration	Klient finns	<code>server/osrm-client.ts</code> finns men OSRM-server måste hostas separat.
Fortnox realtidssync	Envägs	Export till Fortnox fungerar, men full tvåvägs-sync saknas.
IoT-signaler	Schema + API	Tabeller och routes finns, men ingen faktisk IoT-enhetskoppling dokumenterad.
Proaktiv försäljning	UI-sida finns	<code>ProactiveSalesPage.tsx</code> finns, men den bakomliggande analysmotorn verkar vara i tidigt skede.

3.3 Vad som saknas helt (relativt presentationens vision)

Gap	Detalj
Explicit lönsamhetsberäkning per kärn	Presentationen visar en tydlig break-even-modell (fast kostnad vs rörlig intäkt per kärn). Kodbasen har KPI:er och ROI-rapporter men inte denna specifika beräkningsmodell med break-even-punkt.
Stressvarning vid övertid	Presentationen varnar för hälsoeffekter vid >40h. Ingen explicit hälso-/stressindikator i systemet — dock finns SLA-risk och kapacitetsvarningar.
Visuell 28h+12h-arbetsfördelning	Den exakta uppdelningen i “produktionstid” och “reserverad tid” (rast, tankning, buffert, service, admin) som presentationen visar finns inte som en dedikerad vy. Veckoplaneraren visar arbetsbelastning men inte med denna granularitet.
Enhetsansvarigs dagliga checklista	Presentationen beskriver specifika checklistor för enhetsansvarig (planera uppdrag, planera underhåll). Det finns checklistmallar (<code>check-list_templates</code>) men inte i kontexten “enhet-sansvarig-roll”.

4. Technical Assessment

4.1 Arkitektur



4.2 Kodkvalitet — Styrkor

Aspekt	Bedömning
Datamodell	Genomtänkt med 148 tabeller, proper relations, index, soft deletes. Hierarkisk objektmodell med ärvning (koncern→brf→fastighet→rum→kär).
Schema-validering	Zod-scheman genomgående, Drizzle-Zod-integration.
Tenant-isolering	Robust middleware, alla queries filtrerade på <code>tenantId</code> . Dedikerade säkerhetstester.
Testning	41 testfiler, inkl. API-tester, E2E (Playwright), tenant-isoleringstester.
Feature flags	Per-tenant modulsystem med paket-nivåer (standard/professional/enterprise).
Optimeringsalgoritmer	Genuint avancerade — OR-Tools CVRPTW med ALNS postoptimering, inte bara enkel TSP.
Dokumentation	Rik intern dokumentation (75+ markdown-filer), API-kontrakt, arkitekturbeslut.
Errorhantering	AppError-klass, retry med backoff i AI-anrop, timeout-skydd i optimeringsjobb.

4.3 Kodkvalitet — Svagheter

Aspekt	Bedömning
Monolitisk storage	<code>server/storage.ts</code> är 7 464 rader — en “god object” som hanterar allt. Borde brytas upp.
Schema-storlek	<code>shared/schema.ts</code> är 5 557 rader i en fil. Svårt att navigera och underhålla.
TypeScript <code>any</code>	Dokumenterat som känt problem (Task #20) — inte åtgärdat.
Kodduplicering	Dokumenterat i “avduplicering rensningslogg” — pågående arbete med konsolidering.
Replit-beroende	Auth-systemet baserat på Replit-integration (<code>replit_integrations/</code>). Portabilitet till annan hosting kräver omskrivning.
Ingen containerisering (utom optimering)	Huvudapplikationen saknar Dockerfile. Optimeringstjänsten har en, men övrig deploy är Replit-specifik.
Konfigurationskomplexitet	Många miljövariabler och externa tjänsteberoenden (OpenAI, Geoapify, Fortnox, Twilio, Google Cloud Storage).

4.4 Säkerhet

-  Tenant-isoleringsmiddleware med tester
-  Rate limiting (`express-rate-limit`)
-  Lösenordshashning (`bcryptjs`)
-  Session-hantering med PostgreSQL-store
-  `threat_model.md` — dokumenterad hotmodell
-  API-versionering (`/api/v1/`)
-  Auto-tilldelning av tenant avaktiverad i produktion (tidigare säkerhetsbrist dokumenterad)
-  Intern admin-token läses från fil (`.local/.import_token`) — risk vid felkonfiguration

4.5 Skalbarhet

-  Asynkron optimeringskö för stora jobb (>30 stopp)
-  Distance cache-tabell för att undvika upprepade API-anrop
-  Dashboard cache-service
-  Feature flag-cache med TTL
-  Singel Express-process (ingen klusterning/workers dokumenterad)
-  Alla schedulers körs in-process (anomaly monitor, predictive, SLA-risk, geocode, etc.)

5. Rekommendationer

5.1 Brygga mellan presentation och system

#	Rekommendation	Prioritet
1	Skapa en "Enhetsansvarig Dashboard" — en dedikerad vy som matchar slide 5 med plan-vs-utfall-tabell, avvikelseflöde och veckomötesstöd. Data finns redan i KPI-endpoints.	● Hög
2	Implementera break-even-visualisering — en widget i dashboarden som visar daglig progress mot lönsamhetsströskeln (t.ex. 90 av 100 kär!) med färgkodning (röd/gul/grön) som i slide 2.	● Hög
3	Lägg till 28h+12h-veckooöversikt — en vy som visar planerad produktion vs reseravid per vecka, matchande slide 4:s modell. <code>plan-ning_parameters</code> -tabellen kan utökas med dessa värden.	● Medel
4	Carry-over-varning — automatisk notis när dagsmålet inte nås som visar konsekvensen för nästa dag (extra-belastning, stressrisk). Logiken finns implicit men behöver explicit UX.	● Medel
5	Rollen "enhetsansvarig" — överväg att lägga till som en rollnivå mellan planner och user, med begränsad vy fokuserad på det egna teamet/enheten.	● Låg

5.2 Tekniska rekommendationer

#	Rekommendation	Prioritet
6	Bryt upp <code>storage.ts</code> — dela in i domänspecifika services (CustomerStorage, OrderStorage, ResourceStorage). 7 464 rader i en fil är ohållbart.	● Hög
7	Dela <code>schema.ts</code> — gruppera i logiska moduler (core, planning, billing, portal, iot).	● Hög
8	Containerisera hela stacken — skapa Docker Compose med Express-server, PostgreSQL, OR-Tools-service, och valfritt OSRM. Minskar Replit-beroendet.	● Hög
9	Eliminera TypeScript any — Task #20 har status “● PROPOSED” sedan start. Prioritera åtminstone i kritiska flöden (auth, tenant, optimering).	● Medel
10	ML-pipeline till produktion — ml_feature_snapshots skrivs redan. Nästa steg: automatiserad träning, shadow-deploy, A/B-test mot OR-Tools-baseline.	● Medel

5.3 Strategiska rekommendationer

#	Rekommendation
11	Presentationen behöver uppdateras — den beskriver verksamhetsprocessen men nämner inget om det digitala systemets kapacitet. Traivo One har funktionalitet värd att visa (AI, ruttoptimering, kundportal, mobil-app). En ny presentation som inkluderar systemets kapacitet skulle stärka affärscaset.
12	Pilotfokus — Kinab är tydligt pilotkund (docs refererar till “kinab-pilot-startguide”, “kinab-prod-slim-migration”). Fokusera på att leverera de funktioner som presentationen lovar perfekt innan bredare utrullning.
13	Teknisk skuld — kodbasen har vuxit snabbt (221 000 rader, 148 tabeller) med tydliga spår av snabb iteration (namnbyten Nordnav→Traivo→Plannix→Traivo, många “attached_assets” från Replit-sessioner). En konsolideringsfas rekommenderas innan nästa stora funktionsökning.

6. Slutsats

Traivo One-kodbasen är avsevärt mer ambitiös och omfattande än det som presentationen beskriver. Presentationen fokuserar på den operativa verkligheten — hur man planerar dagligt arbete för att vara lönsam — medan systemet adresserar hela den digitala infrastrukturen för att möjliggöra detta.

De koncept som presentationen lyfter fram (daglig måluppfyllelse, veckoplanering, resursbalans, uppföljning) finns alla representerade i kodbasen, men ibland i en mer generisk eller teknisk form snarare än den konkreta, lättförståeliga visualisering som presentationen erbjuder.

Den viktigaste diskrepansen är inte att funktioner saknas — tvärtom finns det för mycket — utan att systemets UX inte fullt ut speglar den operativa modell som presentationen beskriver. En dedikerad “Enhetsansvarig-vy” som kombinerar dagsmål, break-even-kurva, plan-vs-utfall, och vec-komötesagenda skulle vara det mest värdeskapande nästa steget för att brygga gapet.

Tekniskt är systemet imponerande i sin bredd men bär tecken på att ha vuxit snabbt under en prototypningsfas. Konsolidering (monolitisk storage, schemauppdelning, containerisering) bör prioriteras parallellt med ny funktionsutveckling.